

Analysis-oriented Metadata for Data Lakes

Yan Zhao
IRIT, (CNRS/UMR 5505)
Université Toulouse 1 Capitole
Centre Hospitalier Universitaire
(CHU) de Toulouse
Toulouse, France
yan.zhao@irit.fr

Julien Aligon
IRIT, (CNRS/UMR 5505)
Université Toulouse 1 Capitole
Toulouse, France
julien.aligon@ut-capitole.fr

Gabriel Ferretini
IRIT, (CNRS/UMR 5505)
Université Toulouse 1 Capitole
Toulouse, France
gabriel.ferretini@irit.fr

Imen Megdiche
IRIT, (CNRS/UMR 5505)
Institut National Universitaire
Jean-François Champollion, ISIS
Toulouse, France
imen.megdiche@irit.fr

Franck Ravat
IRIT, (CNRS/UMR 5505)
Université Toulouse 1 Capitole
Toulouse, France
franck.ravat@irit.fr

Chantal Soulé-Dupuy
IRIT, (CNRS/UMR 5505)
Université Toulouse 1 Capitole
Toulouse, France
chantal.soule-dupuy@ut-capitole.fr

ABSTRACT

Data lakes are supposed to enable analysts to perform more efficient and efficacious data analysis by crossing multiple existing data sources, processes and analyses. However, it is impossible to achieve that when a data lake does not have a metadata governance system that progressively capitalizes on all the performed analysis experiments. The objective of this paper is to have an easily accessible, reusable data lake that capitalizes on all user experiences. To meet this need, we propose an analysis-oriented metadata model for data lakes. This model includes the descriptive information of datasets and their attributes, as well as all metadata related to the machine learning analyzes performed on these datasets. To illustrate our metadata solution, we implemented an application of data lake metadata management. This application allows users to find and use existing data, processes and analyses by searching relevant metadata stored in a NoSQL data store within the data lake. To demonstrate how to easily discover metadata with the application, we present two use cases, with real data, including datasets similarity detection and machine learning guidance.

CCS CONCEPTS

• **Information systems** → *Database design and models.*

KEYWORDS

Data Lake, Metadata Model, Analysis-oriented Metadata

ACM Reference Format:

Yan Zhao, Julien Aligon, Gabriel Ferretini, Imen Megdiche, Franck Ravat, and Chantal Soulé-Dupuy. 2021. Analysis-oriented Metadata for Data Lakes. In *25th International Database Engineering & Applications Symposium (IDEAS*

2021), July 14–16, 2021, Montreal, QC, Canada. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3472163.3472273>

1 INTRODUCTION

Data Lakes (DL) have emerged as a new solution for Big Data analytics. The ambition of DL is to offer various capacities such as data ingestion, data processing and data analysis. A DL ingests raw data from various sources, stores data in their native format and processes data upon usage [19]. It can also ensure the availability of processed data and provide accesses to different types of end-users, such as data scientists, data analysts and BI professionals.

The major pitfall of data lakes is that they can easily turn into data swamps (i.e. a simple storage space containing all data without any explicit information on them). On the contrary, data analysts or other users need to retrieve and reuse the datasets, associated processes or analyses. To ensure an efficient capitalization of DL, a centralized metadata management system must be deployed. In the literature, several authors propose DL metadata classifications [3, 8, 26], and different metadata systems have been implemented [2, 5, 9]. However, these solutions mainly focus on data management and do not offer metadata on different stages of DL (ingestion, processing and analysis). Regarding data analysis, different models of machine learning (ML) or data mining analysis are proposed by W3C [6] and other authors [12, 18]. These solutions can support data analysts to make better choices on data mining process for one specific dataset, nevertheless, they mainly focus on the phase of data analysis and do not take advantage of data and data processing information across all datasets in data lakes.

Surprisingly, although the ultimate goal of DL is to facilitate data analytics, few solution is proposed to improve the effectiveness and efficiency of the analysts' work through metadata dedicated to support decisional analysis. Therefore, the objective of this paper is to address this lack. For this purpose, information produced during different phases of data analytics has to be taken into account. In particular, these metadata should include: (i) descriptive information about datasets (statistics on attribute values, management of missing values, relationships between datasets and relationship between attributes) and (ii) analytical information about datasets

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IDEAS 2021, July 14–16, 2021, Montreal, QC, Canada

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8991-4/21/07... \$15.00

<https://doi.org/10.1145/3472163.3472273>

(performance of implementation and parameters of the executed algorithms).

The contribution of this paper is to propose a metadata model which contains the information produced in the different phases of data analytics in DL and which can facilitate data analysis tasks. The paper is organized as follows. Section 2 details previous works about metadata in an analysis-oriented purpose. In particular, we detail previous works about metadata classifications found in the Knowledge Discovery Database processes (KDD) context [3], the meta-learning context [21] and the data mining context [6, 12, 18]. Section 3 details the metadata model dedicated to data analysis that includes descriptive and analytical information. In Section 4, we detail three algorithms of metadata feeding and we spotlight two use cases of data analysis based on real datasets.

2 RELATED WORKS

Metadata management is essential for data lakes, different authors provide solutions with different emphases. Datasets metadata, including information of each single dataset and the relationships among them, are studied by various authors [2, 4, 13, 24]. Data processing metadata, aiming to improve the efficacy of data preparation, are also studied by different authors [16, 28]. However, to the best of knowledge, data analysis metadata are slightly studied to better manage data lakes.

We have emphasized the importance of metadata management for data lakes in [19] and proposed a basic metadata model in [20], nevertheless, the metadata model is a simple model which mainly focuses on datasets and the analysis-oriented and data processing metadata are not deeply studied. We have completed data processing metadata in [16], and we address in this paper the analysis metadata aspect for our metadata model.

In the literature, the challenge of metadata for data analysis is particularly studied in the field of meta-learning. Meta-learning aims to learn from past experiences [10], in order to improve the effectiveness of ML algorithms. In this paradigm, metadata describe learning tasks (algorithms) and previously learned models (results). Those metadata concern, for instance, hyper-parameters of predictive models, pipelines of compositions or also meta-features. In the literature, two work axes stand out: (a) the first axis is focused on techniques learning from metadata, for example, by transfer learning; in particular, the recent review of [10] summarizes the different techniques that can be investigated, (b) the second axis is focused on using metadata in different application contexts. For example, we find approaches better supporting the model selection [14] or the recommendation of predictive model hyperparameters [15, 22]. Other recent scenarios are emerging in the context of XAI (eXplainable Artificial Intelligence) such as the approach of [27] which proposes a method to explain how meta-features influence a model performance and expose the most informative hyperparameters.

In a high-level view of this work, a lack of consensus emerges on which metadata to use, not only in the context of ML, but also in data analysis more generally. The works that defines the classification of those metadata in a formal setting are scarce. To the best of our knowledge, the works of [3, 12, 18, 23] and a W3C ML model [6] investigate a classification of metadata related to data analysis.

The authors of [3] give a general overview of different types of necessary metadata to support a KDD process. They globally identify the roles and types of the KDD metadata. In particular, they propose a metadata classification taxonomy covering different types of metadata: general measures, statistical, landmarking measures. However, the classification of metadata is defined in a high level, it needs to be detailed. Moreover, dataset is one of the main element for data analysis, no specific metadata is given to better characterize it. On the other hand, the authors of [23] propose a classification of metadata including simple, statistical, information-theoretic, model-based and landmarking information. In addition to their classification, they propose an R package named MFE¹. This paper has the disadvantage of focusing only on dataset features and it limits its contribution to a classification of metadata without modeling. Paper [3] is centered at the conceptual level whereas the paper of [23] is situated at the technical level. Regarding these limitations, our objective is to cover both conceptual and implementation levels.

Regarding ML model, different ontologies of data mining are proposed [12, 18]. [12] contains detailed descriptions of data mining tasks, data, algorithms, hyperparameters and workflows. [18] defines data mining entities in three layers: specification, implementation and application. The W3C ML model [6] is oriented to ML process. Nevertheless, the relationships between datasets and attributes are not considered. However, one of the important advantages of data lakes is to cross multiple sources of data, so that the relationships between these data are crucial. Hence, adding relationship metadata is inevitable in order to facilitate the work of analysts by selecting and querying relevant datasets and corresponding attributes. Our model contains not only data modeling and algorithm metadata, but also the metadata generated during the data collection and preparation phases, including characteristics, statistical metadata, and relationships between datasets and between attributes. This information is essential to deal with the Feature Selection (FS) [25] problem. It also facilitates data analysis tasks by allowing users to broaden their perspective on the data by viewing the results of performed analyses [1]. A detailed description of our model is discussed in the next section.

3 A GENERIC MODEL OF ANALYSIS-ORIENTED METADATA

The process of data analytics can be described by different phases: business requirement, data exploration, data preparation, modeling/algorithms and data product [11, 17]. In order to simplify data analytics, we propose to manage the metadata which are applied mainly on the following phases : (i) **Data exploration**, which is about collecting and exploring the data that users need. (ii) **Data preparation**, which is about transforming, filtering and cleaning data according to users needs. (iii) **Modeling/algorithms**, which consists in choosing a modeling technique, building a model and evaluating the built model.

To facilitate the data exploration phase, metadata should help users to find datasets that match the analysis objectives or suggest relevant datasets to enrich the analysis. To facilitate the data

¹<https://CRAN.R-project.org/package=mfe>

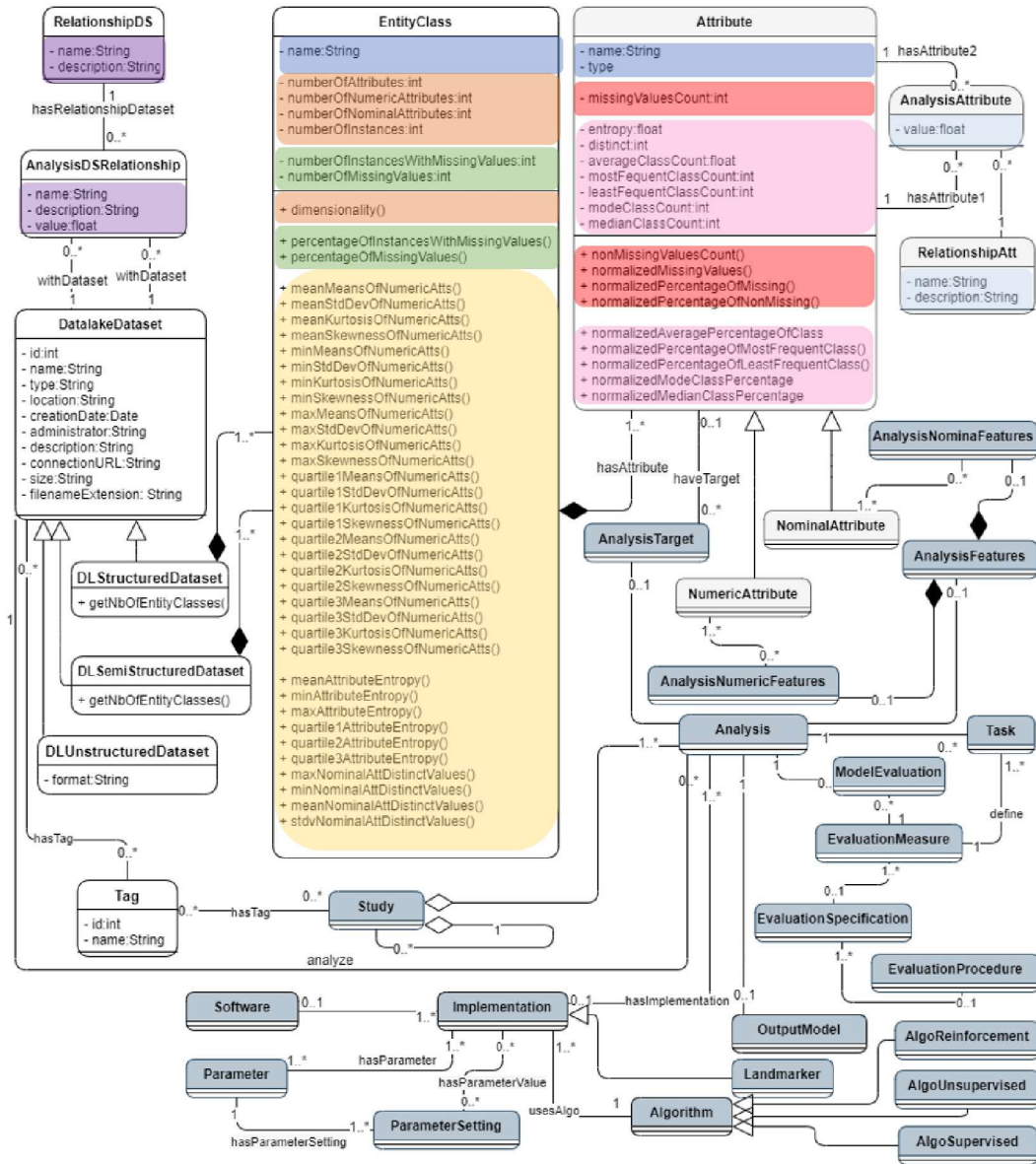


Figure 1: Metadata on datasets and attributes

preparation phase, metadata should present the data characteristics to help users to understand the nature of data and to choose the features of data modeling. Moreover, process characteristics, process definition, technical and content information of data processes should be recorded to help other users understand how data are prepared. To facilitate data modeling phase, metadata should present to users all the existing analyses and certain landmarker results that are carried out on the selected dataset to help users to choose the most appropriate algorithms.

To answer the above mentioned requirements, we extend our previous basic metadata model [20] with the metadata that capitalize the full experiences of data analytic. The complementary metadata can be classified into three categories: datasets, attributes

and analytical metadata. In what follows, we give more details about the different types of metadata in our model. Note that in this paper, we only introduce the analysis-oriented metadata, a more detailed metadata model is available on GitHub².

3.1 Metadata on Datasets

Metadata on datasets allow users to have a general vision of a dataset and can facilitate all data exploration, preparation and modeling phases. They meet the requirements of data analysts who need to know, among other things, (i) dataset structure / schema, which

²https://github.com/yanzhao-irit/data-lake-metadata-management-system/blob/main/images/metadata_model_complet.png

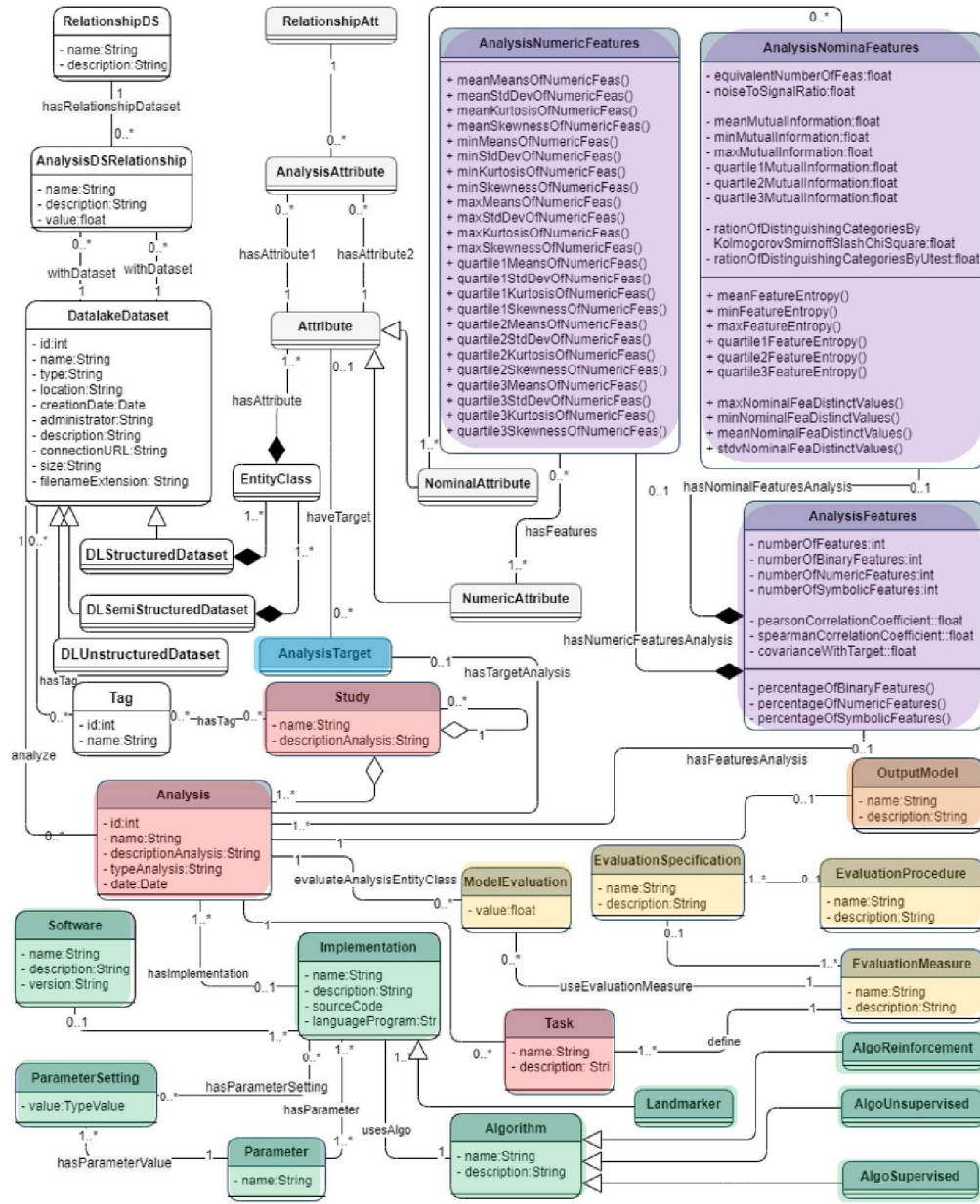


Figure 2: Metadata on data analysis

presents the attributes in a dataset and their type; (ii) dataset dimension, which is useful when the limit of data volume is required; (iii) dataset completeness, for the reason that particular algorithms can only be executed with no missing values; (iv) distribution of attribute values, to fit the algorithms for which better performance is only possible when the instances of the dataset respect certain distributions; and (v) the relationship to other datasets, for instance, similarity or dissimilarity measure values can help them to decide whether or not to apply the same type of learning algorithm on another dataset.

To answer to the above requirements, for each dataset, we provide 5 types of metadata of datasets (see the white classes in Fig. 1): (i) **Schema** metadata (marked in blue) concern the name and type of attributes in an entity class. (ii) **Dimension** metadata (marked in orange) concern the number of attributes, instances and the dimensionality (number of attributes divided by number of instances). (iii) **Missing value** metadata (marked in green) related to the number and percentage of missing values as well as the number and percentage of instances containing missing values. (iv) The **distribution of values** is expressed by 34 metadata (marked in yellow),

such as attribute entropy, kurtosis of numeric attributes and standard deviation of nominal attributes. (v) **Relationships** (marked in purple) with other datasets refer to relationships that can be established by users, such as similarity/dissimilarity and correlation. And users can define their own algorithms for different relationships calculating.

3.2 Metadata on Attributes

Metadata on attributes allow users to have detailed information of attributes for the chosen dataset and these metadata can also facilitate all data exploration, preparation and modeling. We propose the attribute type, completeness, distribution of values and attributes relationship metadata to answer to the same user requirements at the attribute level. For instance, when users analyze a dataset, they may need to choose particular types of attributes for feature engineering, they may need to choose attributes containing no missing values for predictive models, or when dimensionality reduction approaches (Principal Component Analysis (PCA)) are employed, users need to know the correlation or covariance indicators.

Therefore, for each attribute, we provide the metadata (see the gray classes in Fig. 1) of (i) its **type** (classes *NominalAttribute* and *NumericAttribute*), (ii) the **missing values** (marked in red) which consist of the number of missing values, the number of non-missing values, the normalized missing values count and their percentages, (iii) the **distribution of values** (marked in pink), such as the entropy, distinct values, Kurtosis of the numerical attribute and whether the values of a nominal attribute follow a discrete uniform distribution, and (iv) the **relationships** between two different attributes (marked in light blue), for which we can devise our own relationships such as the Spearman's rank correlation, Pearson's correlation and mutual information.

3.3 Metadata on Data Analysis

As far as machine learning is concerned, especially supervised ML, metadata on datasets and attributes are useful but not sufficient. Indeed, the same dataset can be used for different analyses with different classes or features. Moreover, a user can also be interested in landmarker information which is obtained from performing basic learning algorithms. In addition, it is important to help users to obtain information on previous analyses, as existing analyses can be directly reused or contribute to improve current analyses. Thus, analytical metadata have to ensure collaborative capabilities.

Therefore, to complete our metadata model, we add analytical metadata (see the dark blue classes in Fig. 2) to help users find and possibly reuse existing analyses with their business objectives, selected features and target class, implemented parameters, output model and evaluation of the result : (i) the **study** metadata (marked in red) which is a project containing different analysis, (ii) the **feature analysis** (marked in purple) which concerns the number of features type, the relationships with target attribute and the distribution of all the feature values, (iii) the **target attribute analysis** (marked in blue) in the case of supervised ML, (iv) the **implementation** (marked in green) which is carried out for the analyses, and (v) the **model** of an analysis (marked in orange). (vi) the **evaluation of analysis** (marked in yellow) with different measures. Note that in this paper, we define the class attribute of

supervised analyses as the target attribute, all other attributes are features.

4 IMPLEMENTATION AND USE CASE VALIDATION

The proposed metadata can be used in decision support for data exploration, data preparation (including feature engineering) or the use of model/algorithms. The metadata facilitates (i) the choice of the most relevant datasets and their understanding as well as the feature engineering and (ii) the verification of previous analyses to better prepare and optimize the work.

We chose to use a graph database (Neo4j) to store these metadata for the reason that: (i) a good flexibility can be ensured, (ii) it is easier to query a graph database when the search depth is important, and (iii) some machine learning algorithms are integrated which can be useful to build a recommender system³. Note that the proposed metadata model can also be stored with other technologies, for instance, if relational database is chosen, for the following three algorithms, all the *properties* become *attributes*, all the *nodes* become *instances of the corresponding class table*, and all the *relationships* become *instances of the relationship table*.

This metadata database is integrated in the data lake system and is managed through an application⁴. In the following subsections, we explain how to feed metadata via an application into a graph database by three algorithms. Then we expose two use cases that demonstrate the contribution of our model compared to other models in the literature.

4.1 Metadata feeding

The metadata that we introduced previously are managed by a locally deployed application that interfaces with user input and calls three algorithms to instantiate the Neo4J database according to our metadata model.

In this application, users can input descriptive information, for instance, name and description of datasets or processes, chosen algorithms and set parameters, with an interface. The application can generate automatically other metadata, for instance, schematic metadata of datasets and relationships between different datasets to facilitate future analyses.

During data preparation phase, when a new dataset is stored in the DL (by data ingestion or processing), the application generates descriptive metadata of dataset and its attributes automatically (see Algo. 1).

During data analysis phase, for each study, users can firstly choose features and the target attribute to let the system calculate statistical metadata of features and run landmarkers for machine learning guidance (see Algo. 2). With the information that the system pre-analyzed, the user can implement different algorithms with different parameter settings. For each algorithm with a parameter setting, an *Analysis* and an *Implementation* are instantiated (see Algo. 3).

³<https://neo4j.com/product/graph-data-science-library/>

⁴<https://github.com/yanzhao-irit/data-lake-metadata-management-system>

Algorithm 1: DatasetAttributesMetadata

```

Input: connectionURL, descriptionDS
Data: newDS
/* use a table to store properties */
1  datalakeDatasetProperties ← createProperties('description',
    getDatasetName(newDS), 'connectionURL', 'connectionURL', 'size',
    getFileInfo(newDS))
2  datalakeDataset ← createNode('DatalakeDataset',
    datalakeDatasetProperties)
/* store schematic metadata */
3  datasetStruct ← getDatasetStructurality(connectionURL)
4  if (datasetStruct.type = "structured" or "semi-structured") then
5      entities[] ← getEntityClasses(newDataset)
6      foreach e ∈ entities do
7          atts[] ← getAttributes(e)
8          entityClassProperties ← createProperties('name', getEntityName(e),
            getEntityStatistics(atts[]))
            // function getEntityStatistics() returns an array including the name
            and value of each statistical metadata
9          entityClass ← createNode('EntityClass', entityClassProperties)
10         createRelation('DatalakeDataset-EntityClass',
            datalakeDataset, entityClass)
11         foreach att ∈ atts[] do
12             if getAttType(att) = 'numeric' then
13                 numericAttributeProperties ← createProperties('name',
                    getAttName(att), getNumericAttStat(att)) attribute ←
                    createNodeNeo4j('NumericAttribute',
                    numericAttributeProperties)
14             else
15                 nominalAttributeProperties ← createProperties('name',
                    getAttName(att), getNominalAttStat(att))
                    attribute ← createNode('NominalAttribute',
                    nominalAttributeProperties)
16             createRelation('EntityClass-Attribute', entityClass, attribute)
17         /* For each predefined RelationshipAtt we calculate the value of
            relationship between attributes */
18         analysisAttributes[] ← getAnalysisAttribute(atts[],
            relationshipAtts[])
19         foreach an ∈ analysisAttributes[] do
20             relationArr ← createNode('AnalysisAttribute',
                createProperties('value', an.value))
                createRelation('AnalysisAttribute-Attribute',
                relationArr, an.attribute1)
21             createRelation('AnalysisAttribute-Attribute',
                relationArr, an.attribute2)
22             createRelation('AnalysisAttribute-RelationshipAtt',
                relationArr, an.relationshipAtt)
23         else
24             addAttToNeo4jNode(datalakeDataset, getDatasetFormat(newDS))
25             /* For each predefined RelationshipDS we calculate the value of relationship
                between datasets */
26             analysisDSRelationships[] ←
                getAnalysisDSRelation(datalakeDataset, datalakeDatasets[], relationshipDSs[])
27             foreach anDs ∈ analysisDSRelationships[] do
28                 relationDs ← createNode('AnalysisDSRelationship',
                    createProperties('value', anDs.value))
29                 createRelation('AnalysisDSRelationship-DatalakeDataset',
                    relationDs, anDs.datalakeDataset1)
30                 createRelation('AnalysisDSRelationship-DatalakeDataset',
                    relationDs, anDs.datalakeDataset2)
31                 createRelation('AnalysisDSRelationship-RelationshipDS',
                    relationDs, anDs.relationshipDS)

```

Algorithm 2: Pre_analysis

```

Input: studyName, studyDesc, datalakeDataset, targetAtt, features[],
    analysisName, analysisDesc
Data: selected dataset ds
/* calculate statistics for analysis */
1  Function statisticsAnalysis(study, datalakeDataset, targetAtt,
    features[], analysisName, analysisDesc):
2      analysis ← createAnalysis('Analysis', createProperties('name',
        analysisName, 'description', analysisDesc))
3      createRelation('Analysis-DatalakeDataset', analysis, datalakeDataset)
4      createRelation('Analysis-Study', analysis, study)
5      analysisTarget ← createNode('AnalysisTarget')
6      createRelation('Analysis-AnalysisTarget', analysis, analysisTarget)
7      createRelation('AnalysisTarget-Attribute', analysisTarget, targetAtt)
8      analysisFeatures ← createNode('AnalysisFeatures',
        createProperties(getStatAnalysisFeatures(features[])))
9      createRelation('Analysis-AnalysisFeatures', analysis, analysisFeatures)
10     analysisNumericFeatures ← createNode('AnalysisNumericFeatures',
        createProperties(getStatAnalysisNumericFeatures(features[])))
11     createRelation('AnalysisNumericFeatures-AnalysisFeatures',
        analysisNumericFeatures, analysisFeatures)
12     analysisNominalFeatures ←
        createNodeNeo4j('AnalysisNominalFeatures',
        createProperties(getStatAnalysisNominalFeatures(features[])))
13     createRelation('AnalysisNominalFeatures-AnalysisFeatures',
        analysisNominalFeatures, analysisFeatures)
14     return analysis
    /* run predefined landmarks */
15 Function
    runLandmarks(study, datalakeDataset, targetAtt, features[]):
16     landmarks[] ← getLandmarks()
17     foreach lm ∈ landmarks[] do
18         analysis ←
            statisticsAnalysis(study, datalakeDataset, targetAtt, features[], lm.name,
            lm.description)
19         output ← runAlgo(analysis, lm)
20         outputModel ← createNode('OutputModel', createProperties('name',
            'description', output))
21         createRelation('Analysis-OutputModel', analysis, outputModel)
22         foreach m ∈ getEvaluationMeasures() do
23             modelEvaluation ← createNode('ModelEvaluation',
                createProperties('value', evaluate(m, analysis)))
24             createRelation('Analysis-ModelEvaluation',
                analysis, modelEvaluation)
25             createRelation('ModelEvaluation-EvaluationMeasure',
                modelEvaluation, m)
26
27     study ← createNode('Study', createProperties('name', studyName,
    'description', studyDesc))
28     runLandmarks(study, datalakeDataset, targetAtt, features[])

```

Algorithm 3: Analyse_dataset

Input: *study*, *analysisName*, *analysisDesc*, *datalakeDataset*, *targetAtt*, *features*[], *sourceCode*, *algo*, *paras*[], *paraSets*[]

Data: selected dataset *ds*

```

1 analysis ← statisticsAnalysis(study, datalakeDataset, targetAtt,
  features[], analysisName, analysisDesc)
2 implementation ← createNode('Implementation',
  createProperties('sourceCode', sourceCode))
3 createRelation('Implementation-Analysis', implementation, analysis)
4 algo ← createNode('Algorithm', createProperties('name', algo.name,
  'description', algo.description))
5 createRelation('Implementation-Algorithm', implementation, algo)
6 foreach para ∈ paras[] do
7   parameter ← createNode('Parameter', createProperties('name', para))
8   createRelation('Implementation-Parameter', implementation, parameter)
9 foreach paraSet ∈ paraSets[] do
10  parameterSetting ← createNode('ParameterSetting',
    createProperties('value', paraSet.value))
11  createRelation('Implementation-ParameterSetting',
    implementation, parameterSetting)
12  createRelation('ParameterSetting-Parameter', parameterSetting,
    getParameter(paraSet.para))
13 output ← runAlgo(analysis, algo)
14 outputModel ← createNode('OutputModel', createProperties('name', ,
  'description', output))
15 createRelation('Analysis-OutputModel', analysis, outputModel)
16 foreach m ∈ getEvaluationMeasures() do
17  modelEvaluation ← createNode('ModelEvaluation',
    createProperties('value', evaluate(algo, analysis)))
18  createRelation('Analysis-ModelEvaluation', analysis, modelEvaluation)
19  createRelation('ModelEvaluation-EvaluationMeasure',
    modelEvaluation, m)

```

4.2 Use Case Validation

In this section, we illustrate two examples based on real-life situations to show the utility of stored metadata.

As a running example, a data analyst in biology, named Bob, would like to identify the indicators having an important impact on the colon cancer. For this purpose, he uses the *CHSI*⁵ dataset including more than 200 health indicators for each of the 3,141 United States counties. A data engineer constructs three extracts of the dataset to prepare an analysis of three cancers (colon, breast and lung) from this initial dataset according to the objective of binary classification (to have or not the cancer type). Indeed, due to the high number of indicators dedicated to multiple types of diseases (especially on cancers), an analysis can be difficult to be performed directly in a single dataset.

These datasets have six indicators oriented to the analysis of the *colon*, *breast* and *lung* cancers, respectively, of people with the measures of *obesity*, *high blood pressure* and *smoker*. There are 3,141 rows in the dataset and each of them represents a group of people living in a same *county* for each *state* of the United States.

4.2.1 Use Case 1: Dataset similarity detection. Identifying the right ML model for a given dataset can be a very tedious task. On this way, detecting similar datasets where previous models have already been executed is a possible solution. This would make it easier for

a user to identify the type of algorithm that can be launched for their dataset. This approach is inspired by the automated ML field where a model is automatically proposed from a set of existing datasets and ML workflows [7]. Thanks to the dataset and attribute metadata, it is easier to detect stored close datasets, as long as a proximity measure has been implemented in the ML (*Relationship* metadata in Fig. 1).

In our running example, a dissimilarity measure, proposed in [21], is applied (Note that other different algorithms can be also generated). In different analysis contexts, users can use different algorithms. This measure is defined on two levels. The first level estimates a dissimilarity between datasets (classes *AnalysisDSRelationship* and *RelationshipDS*) and the second level is between attribute (classes *AnalysisAttribute* and *RelationshipAtt*).

Bob wants to retrieve more datasets which are close to the colon cancer dataset that he analyzes. Thus, he uses the metadata management application to find colon cancer dataset, and goes to the *Relationship Dataset* tab to check the information that he needs (see left image in Fig. 3). He finds out that breast cancer dataset and lung cancer dataset are both similar to the colon cancer dataset (with the dissimilarities <0.01).

Thanks to the result, Bob decides to enrich his analysis with the breast and lung cancer datasets. Moreover, Bob can searching all the analyses/algorithms that are already executed on these datasets. For each algorithm, he can get the information of used algorithm name, parameter names, parameter values and evaluation values (see right image in Fig. 3). For instance, the shown result concerns an analysis of breast cancer, for which a random forest algorithm is preformed with two parameters (*n_estimators* = 10, *test_size* = 0.2) and the result is evaluated by accuracy (0.42).

Looking at these results, Bob decided to perform a Random Forest and SVM algorithms on his dataset because they give the best results in terms of accuracy on the breast (0.48) and lung cancer (0.58) analyses.

4.2.2 Use Case 2: Machine learning guidance. Metadata based on landmarks are a powerful solution to estimate which kind of machine learning model will be performed better on a dataset. By running a set of diverse simple models on the dataset, a baseline of model performances can be established. This baseline is a first indicator of possible performances for more complex versions of each model, which reduces the need for trial and error testing commonly associated with the choice of machine learning model.

In our running example, four landmarks (and derived versions using different parameters) are available in the Data Lake. These landmarks include: Decision Tree, Naive Bayes, KNN and Random Forest (see class *Landmarker* in Fig. 2). They are evaluated thanks to one measure: Error rate.

Bob wishes to execute a predictive model on the colon cancer dataset. To guide him, he wants to know the error rate of the landmarks available in the data lake. In the application, he can check the existing landmarks and evaluations one by one. For instance, the left image of Fig. 4 concerns the landmark *RandomTreeDepth3* and its error rate is 0.4649; the right image of Fig. 4 concerns the landmark *REPTreeDepth3* and its error rate is 0.5070.

After checking all the landmarks that have been executed on the colon cancer dataset, Bob finds the lowest error is performed

⁵<https://healthdata.gov/dataset/community-health-status-indicators-chsi-combat-obesity-heart-disease-and-cancer>

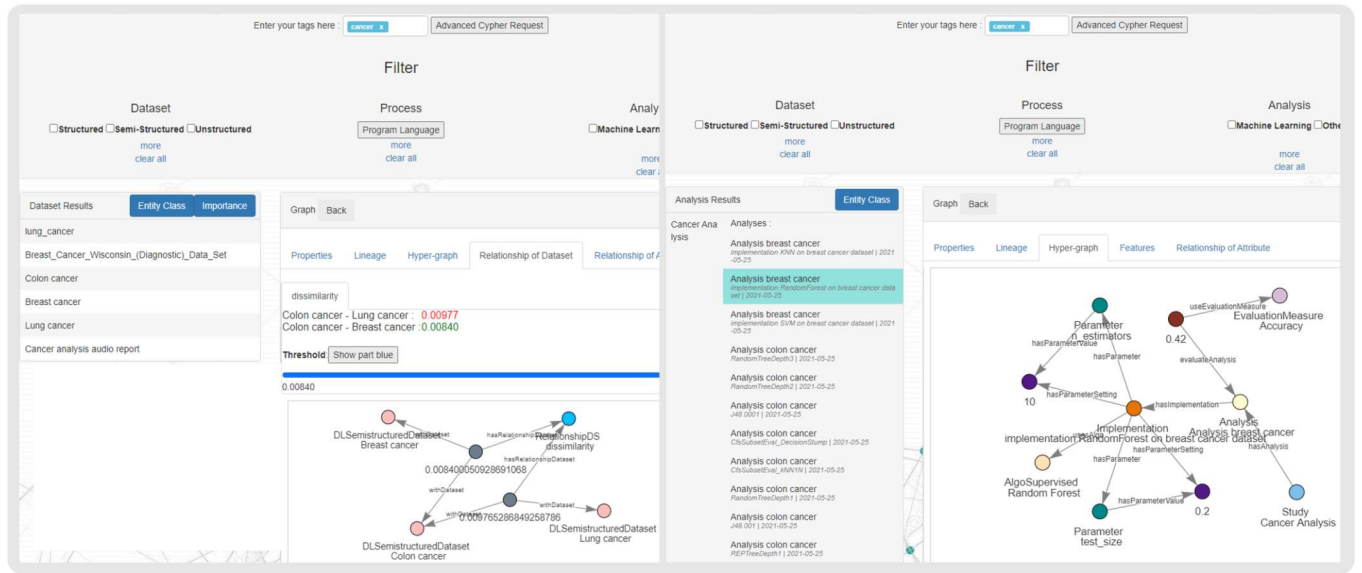


Figure 3: Application results for the use case 1

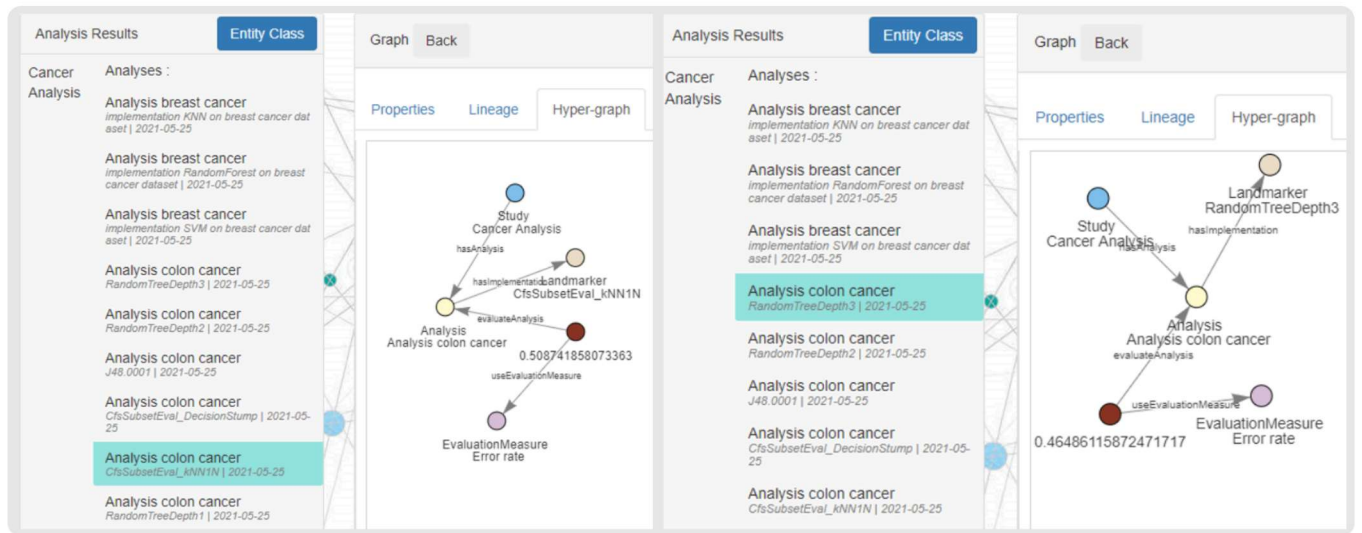


Figure 4: results for the usecase2

by *J48.0001*. Therefore, Bob can use the landmarker *J48* which gives him a basis to build a better model then.

The application has two accesses, one is a graphical interface that dedicated to all users of the data lake (data scientists, statisticians and analysts) for helping them find, access and reuse existing datasets or analyses. For specialists who have the skills of Neo4j, we provide a second access for more rich research with more characteristics. If Bob does not want to click on every landmarker to look for the lowest error rate, he can always do advanced research by writing query by himself. We provide a function of free consultation in the wapplication which requires Cypher (Neo4j querying

language) skill. So that Bob can use the Chyper query below to find all the landmarkers of colon cancer dataset (left result in Fig. 5).

MATCH

```
(ds) -[rhe : hasEntityClass] -> (ec : EntityClass),
(a : Analysis) -[ra : analyze] -> (ds),
(a) -[rhi : hasImplementation] -> (lm : Landmarker),
(evl : ModelEvaluation) -[re : evaluateAnalysis] -> (a),
(evl) -[ru : useEvaluationMeasure] -> (m)
```

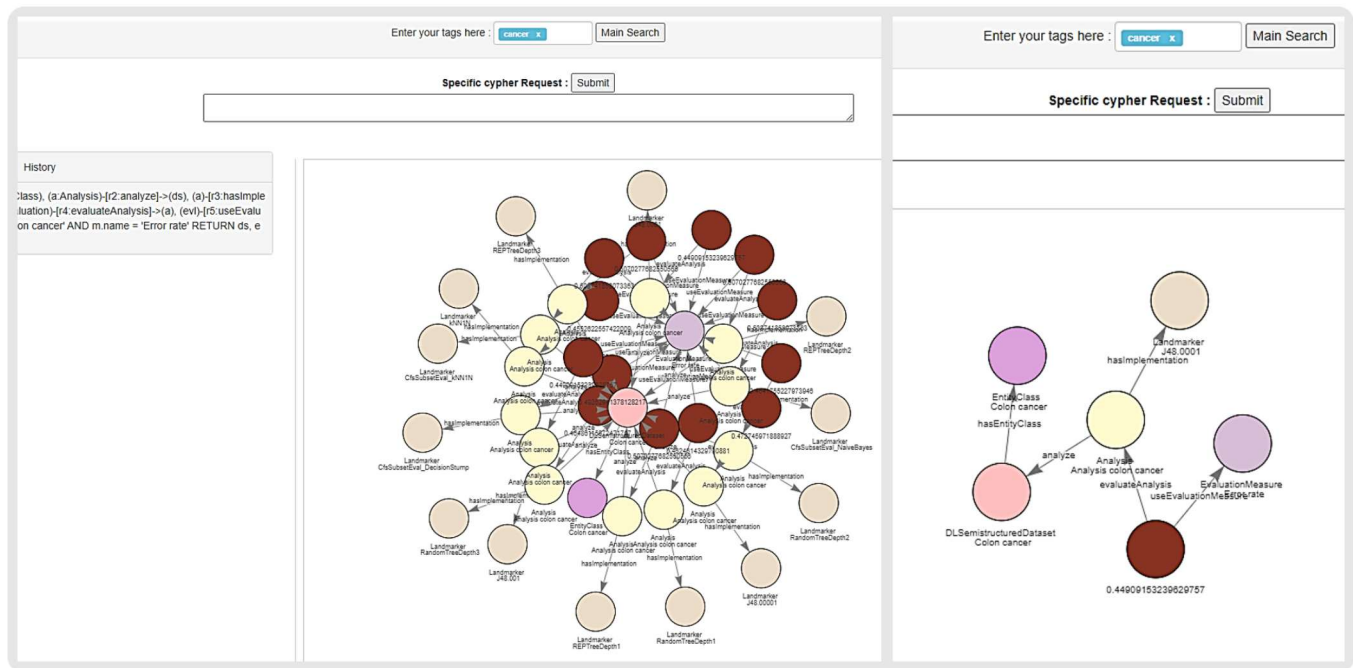


Figure 5: Query and result of the free search

WHERE

```
ds.name = 'Colon cancer'
AND m.name = 'Error rate'
```

```

RETURN ds, ec, a, lm, evl, m, rhe, ra,
        rhi, re, ru

```

By adding one condition on the query, Bob can get the lowest error rate landmarker directly (right result in Fig. 5).

ORDER BY evl.value ASC LIMIT 1

4.3 Use case discussions

Machine learning guidance and finding similar datasets are actual topics in the literature but tackled separately. In this section we introduced the solution to answer these needs through the use cases. Our model covers complete panel of metadata that could be used for different use cases. If we recall the interest of metadata to prevent the data lake from reverting to data swamp, there is inevitably a discussion to be conducted on the time saved by having such a metadata system implemented. Thanks to our metadata model, Bob could automatically access to various datasets and their statistics information as well as the already performed algorithms. During the use case 1, Bob queried pre-calculated correlations during a feature selection task; In the use case 2, the data lake brings up the most relevant algorithms in front of a new dataset. Without the data lake metadata, Bob would have had to manipulate the correlation functions himself or use specific tool or programs for obtaining landmarks results and handling the similarity measures and the datasets. Thus, all of these tedious tasks are made transparent to users. Bob can define his own analysis by devising only a few queries on the data lake to start a machine learning process. This

centralized architecture of metadata facilitates the work of analysis and saves time for Bob.

5 CONCLUSION & PERSPECTIVES

Without an appropriate metadata management, a data lake can easily turn into a data swamp which is invisible, incomprehensible and inaccessible. Different data lake metadata solutions are proposed with different emphases, however, there is no solution that focuses on analysis metadata which allow users to cross different datasets and analyses to take advantage of the existing datasets information and analysis experience in the data lake.

In this paper, we propose a complete solution of analysis-oriented metadata for data lakes which includes a metadata model, three algorithms of the metadata detection and an application which allow users to search metadata easily. The metadata model contains not only descriptive information on the datasets (statistics on attribute values, management of missing values and relationships between datasets and attributes), but also analytical information on performed analyses of these datasets (studies, tasks and implementations and evaluations of analyses, performances and parameters of the previously executed algorithms. The algorithms explains how to automatically detect analysis-oriented metadata from different structural types of datasets and preformed landmarks or analysis. The application allow users to find information of all the elements stored in the data lake in an ergonomic way. To validate our solution, we illustrate two usecases with real data to explain the completeness, feasibility and utility of our solution.

For our future work, we plan to include other metadata dedicated to additional types of analyses (other than ML algorithms) such as statistical or OLAP analyses. Moreover, a recommender system may

suggest to users the most appropriate algorithms or parameters for different analyses. An explainability system can be considered to explain the results of machine learning analyses as well as other types of analyses. Nevertheless, a massively user-oriented experimentation using our metadata model will be also considered.

REFERENCES

- [1] Qasem Al-Tashi, Said Jadid Abdulkadir, Helmi Md Rais, Seyedali Mirjalili, and Hitham Alhussian. 2020. Approaches to Multi-Objective Feature Selection: A Systematic Literature Review. *IEEE Access* 8 (2020), 125076–125096. <https://doi.org/10.1109/ACCESS.2020.3007291>
- [2] Ayman Alserafi, Alberto Abelló, Oscar Romero, and Toon Calders. 2016. Towards Information Profiling: Data Lake Content Metadata Management. In *2016 IEEE 16th International Conference on Data Mining Workshops (ICDMW)*. 178–185. <https://doi.org/10.1109/ICDMW.2016.0033>
- [3] Besim Bilali, Alberto Abelló, Tomàs Aluja-Banet, and Robert Wrembel. 2016. Towards Intelligent Data Analysis: The Metadata Challenge. In *Proceedings of the International Conference on Internet of Things and Big Data*. SCITEPRESS - Science and Technology Publications, Rome, Italy, 331–338. <https://doi.org/10.5220/0005876203310338>
- [4] Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, and Nikolaos Konstantinou. 2020. Dataset Discovery in Data Lakes. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 709–720. <https://doi.org/10.1109/ICDE48307.2020.00067>
- [5] Claudia Diamantini, Paolo Lo Giudice, Domenico Potena, Emanuele Storti, and Domenico Ursino. 2021. An Approach to Extracting Topic-guided Views from the Sources of a Data Lake. *Information Systems Frontiers* 23, 1 (Feb. 2021), 243–262. <https://doi.org/10.1007/s10796-020-10010-x>
- [6] Diego Esteves, Diego Moussallem, Ciro Baron Neto, Tommaso Soru, Ricardo Usbeck, Markus Ackermann, and Jens Lehmann. 2015. MEX vocabulary: a lightweight interchange format for machine learning experiments. In *Proceedings of the 11th International Conference on Semantic Systems (SEMANTICS '15)*. Association for Computing Machinery, New York, NY, USA, 169–176. <https://doi.org/10.1145/2814864.2814883>
- [7] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2 (NIPS'15)*. MIT Press, Cambridge, MA, USA, 2755–2763.
- [8] Neil Foshay, Avinandan Mukherjee, and Andrew Taylor. 2007. Does data warehouse end-user metadata add value? *Commun. ACM* 50, 11 (Nov. 2007), 70–77. <https://doi.org/10.1145/1297797.1297800>
- [9] Alon Halevy, Flip Korn, Natalya F Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. 2016. Managing Google's data lake: an overview of the GOODS system. *IEEE Data Eng. Bull.* 39, 3 (2016), 5–14.
- [10] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren (Eds.). 2019. *Automated Machine Learning: Methods, Systems, Challenges*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-05318-5>
- [11] Charles M. Judd, Gary H. McClelland, and Carey S. Ryan. 2011. *Data Analysis: A Model Comparison Approach, Second Edition*. Routledge.
- [12] C Maria Keet, Agnieszka Ławrynowicz, Claudia d'Amato, Alexandros Kalousis, Phong Nguyen, Raul Palma, Robert Stevens, and Melanie Hilario. 2015. The Data Mining OPTimization Ontology. *Web Semantics: Science, Services and Agents on the World Wide Web* 32, C (May 2015), 43–53. <https://doi.org/10.1016/j.websem.2015.01.001>
- [13] Sven Langenecker, Christoph Sturm, Christian Schalles, and Carsten Binnig. 2021. *Towards Learned Metadata Extraction for Data Lakes*. Gesellschaft für Informatik, Bonn. <https://doi.org/10.18420/btw2021-17>
- [14] Enrique Leyva, Antonio González, and Raúl Pérez. 2015. A Set of Complexity Measures Designed for Applying Meta-Learning to Instance Selection. *IEEE Transactions on Knowledge and Data Engineering* 27, 2 (Feb. 2015), 354–367. <https://doi.org/10.1109/TKDE.2014.2327034>
- [15] Rafael G. Mantovani, André L. D. Rossi, Edesio Alcobaça, Joaquin Vanschoren, and André C. P. L. F. de Carvalho. 2019. A meta-learning recommender system for hyperparameter tuning: Predicting when tuning improves SVM classifiers. *Information Sciences* 501 (Oct. 2019), 193–221. <https://doi.org/10.1016/j.ins.2019.06.005>
- [16] Imen Megdiche, Franck Ravat, and Yan Zhao. 2021. Metadata Management on Data Processing in Data Lakes. In *SOFSEM 2021: Theory and Practice of Computer Science (Lecture Notes in Computer Science)*, Tomáš Bureš, Riccardo Dondi, Johann Gamper, Giovanna Guerrini, Tomasz Jurdziński, Claus Pahl, Florian Sikora, and Prudence W.H. Wong (Eds.). Springer International Publishing, Cham, 553–562. https://doi.org/10.1007/978-3-030-67731-2_40
- [17] Cathy O'Neil and Rachel Schutt. 2013. *Doing Data Science: Straight Talk from the Frontline*. "O'Reilly Media, Inc."
- [18] Panče Panov, Larisa Soldatova, and Sašo Džeroski. 2014. Ontology of core data mining entities. *Data Mining and Knowledge Discovery* 28, 5 (Sept. 2014), 1222–1265. <https://doi.org/10.1007/s10618-014-0363-0>
- [19] Franck Ravat and Yan Zhao. 2019. Data Lakes: Trends and Perspectives. In *Database and Expert Systems Applications (Lecture Notes in Computer Science)*, Sven Hartmann, Josef Küng, Sharma Chakravarthy, Gabriele Anderst-Kotsis, A Min Tjoa, and Ismail Khalil (Eds.). Springer International Publishing, Cham, 304–313. https://doi.org/10.1007/978-3-030-27615-7_23
- [20] Franck Ravat and Yan Zhao. 2019. Metadata Management for Data Lakes. In *New Trends in Databases and Information Systems (Communications in Computer and Information Science)*. Springer International Publishing, Cham, 37–44. https://doi.org/10.1007/978-3-030-30278-8_5
- [21] William Raynaud, Chantal Soule-Dupuy, and Nathalie Valles-Parlangeau. 2016. Meta-mining Evaluation Framework: A Large Scale Proof of Concept on Meta-learning. In *AI 2016: Advances in Artificial Intelligence (Lecture Notes in Computer Science)*, Byeong Ho Kang and Quan Bai (Eds.). Springer International Publishing, Cham, 215–228. https://doi.org/10.1007/978-3-319-50127-7_18
- [22] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *arXiv:1602.04938 [cs, stat]* (Aug. 2016). [arXiv:1602.04938 \[cs, stat\]](https://arxiv.org/abs/1602.04938)
- [23] Adriano Rivoli, Luis P. F. Garcia, Carlos Soares, Joaquin Vanschoren, and André C. P. L. F. de Carvalho. 2018. Characterizing classification datasets: a study of meta-features for meta-learning. *arXiv:1808.10406 [cs, stat]* (Aug. 2018). [arXiv:1808.10406 \[cs, stat\]](https://arxiv.org/abs/1808.10406)
- [24] Tyler J. Skluzacek, Rohan Kumar, Ryan Chard, Galen Harrison, Paul Beckman, Kyle Chard, and Ian T. Foster. 2018. Skluma: An Extensible Metadata Extraction Pipeline for Disorganized Data. In *2018 IEEE 14th International Conference on e-Science (e-Science)*. 256–266. <https://doi.org/10.1109/eScience.2018.00040>
- [25] Saúl Solorio-Fernández, J. Ariel Carrasco-Ochoa, and José Fco. Martínez-Trinidad. 2020. A review of unsupervised feature selection methods. *Artificial Intelligence Review* 53, 2 (Feb. 2020), 907–948. <https://doi.org/10.1007/s10462-019-09682-y>
- [26] Jovan Varga, Oscar Romero, Torben Bach Pedersen, and Christian Thomsen. 2014. Towards Next Generation BI Systems: The Analytical Metadata Challenge. In *Data Warehousing and Knowledge Discovery (Lecture Notes in Computer Science)*, Ladjel Bellatreche and Mukesh K. Mohania (Eds.). Springer International Publishing, Cham, 89–101. https://doi.org/10.1007/978-3-319-10160-6_9
- [27] Katarzyna Woźnica and Przemysław Biecek. 2020. Towards better understanding of meta-features contributions. *arXiv:2002.04276 [cs, stat]* (Feb. 2020). [arXiv:2002.04276 \[cs, stat\]](https://arxiv.org/abs/2002.04276)
- [28] Yi Zhang and Zachary G. Ives. 2020. Finding Related Tables in Data Lakes for Interactive Data Science. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1951–1966. <https://doi.org/10.1145/3318464.3389726>